Dynamic Network Routing Simulator

Graph-Based Shortest-Path Routing with Dynamic Failure Recovery

Project Title	Dynamic Network Routing Simulator
Subject	Advanced Computer Networks & Mobile Computing
Project Type	Network Simulation & Routing Optimization System
Difficulty Level	Advanced
Semester	Semester 7, B.Tech / GSSIPU Curriculum
Author	Mayank Swaraj

Abstract

This paper presents the design and implementation of a Dynamic Network Routing Simulator that models core Internet routing behavior: representing a network as a weighted graph of routers, computing shortest paths between them, maintaining per-router routing tables, and dynamically recalculating routes when a router fails. The simulator draws on the same graph-theoretic foundation used by real-world routing protocols such as OSPF, and reproduces their essential property — automatic failover to an alternative path when part of the network becomes unreachable. This paper documents the system architecture, the routing protocols that motivate the design (RIP, OSPF, BGP), a worked example of shortest-path computation and failure recovery on a four-router network, and a Python implementation of the underlying graph model.

Keywords: network routing, shortest path, graph algorithms, OSPF, RIP, BGP, failure recovery, network simulation

Mayank Swaraj | github.com/mayankswaraj18cr-cmd | linkedin.com/in/mayank-swaraj-4b84b6410 | mayankswaraj18cr@gmail.com

1. Introduction

The Internet functions because routers continuously determine the best available path for forwarding data, and automatically select an alternative path when a router or link along the current path fails. This project, undertaken as a Semester 7 project under Advanced Computer Networks & Mobile Computing, builds a simulator that reproduces this behavior on a small representative network: it models routers and their interconnections as a weighted graph, computes shortest paths using a Dijkstra-based path calculator, maintains a routing table per source router, and recalculates routes dynamically when a router is marked as failed.

2. Routing Protocols

Three routing protocols motivate the simulator's design. The Routing Information Protocol (RIP) is a distance-vector protocol that selects routes based purely on hop count, making it simple but suboptimal when link costs vary. Open Shortest Path First (OSPF) is a link-state protocol that computes the genuinely shortest path using each link's assigned cost, which is the model this simulator follows most closely. The Border Gateway Protocol (BGP) operates at a larger scale, routing traffic between autonomous systems across the wider Internet rather than within a single network. The simulator's path calculator implements OSPF-style shortest-path routing, since it directly parallels the weighted shortest-path problem central to this project.

3. System Architecture

The simulator is structured as a five-stage pipeline, shown in Figure 1. Network nodes (routers) and their weighted connections form the input graph. The routing engine holds this topology and exposes it to a path calculator, which runs a shortest-path algorithm to determine, for any source and destination, the lowest-cost route. The result populates a routing table for each router, which is then consulted during packet transmission to determine the next hop at every step until the packet reaches its destination.

Mayank Swaraj | github.com/mayankswaraj18cr-cmd | linkedin.com/in/mayank-swaraj-4b84b6410 | mayankswaraj18cr@gmail.com

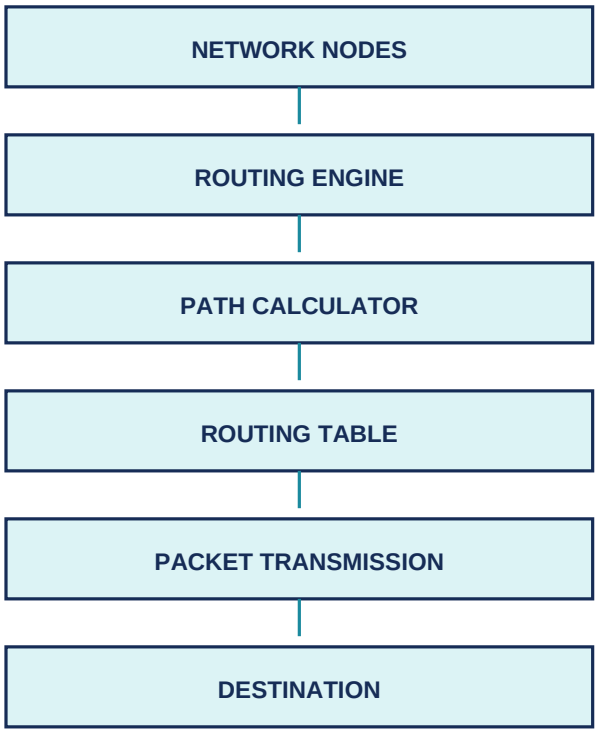

Figure 1. Five-stage routing pipeline from network topology to packet delivery.

4. Example Network

The worked example used throughout this paper is a four-router network — R-A, R-B, R-C, and R-D — connected as shown in Figure 2: R-A–R-B with cost 2, R-A–R-C with cost 4, R-B–R-D with cost 3, and R-C–R-D with cost 1.

Mayank Swaraj | github.com/mayankswaraj18cr-cmd | linkedin.com/in/mayank-swaraj-4b84b6410 | mayankswaraj18cr@gmail.com

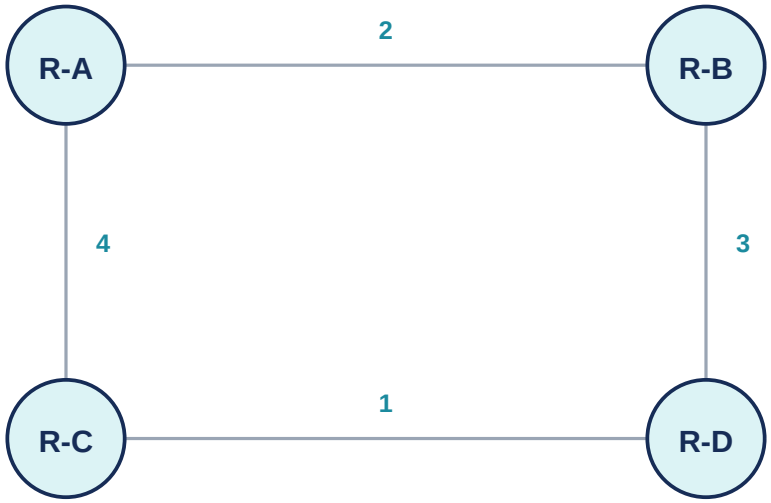

Figure 2. Example four-router network with weighted links.

5. Routing Table Computation

Running the shortest-path calculator from source router R-A produces the routing table shown in Table 1. Note that the shortest path to R-D is via R-B (cost 2 + 3 = 5) rather than via R-C (cost 4 + 1 = 5); in this particular network both routes tie at cost 5, and the table reflects the first-discovered minimum-cost route.

Destination	Next Hop	Cost
Router B	B	2
Router C	C	4
Router D	B	5

Table 1. Routing table computed at Router A under normal (no-failure) conditions.

Mayank Swaraj | github.com/mayankswaraj18cr-cmd | linkedin.com/in/mayank-swaraj-4b84b6410 | mayankswaraj18cr@gmail.com

6. Network Failure Simulation

The simulator's dynamic update feature is demonstrated by marking Router B as failed. All links incident to R-B are removed from the graph, and the path calculator is re-run. Figure 3 shows the resulting topology: with R-B unavailable, the only remaining path from R-A to R-D runs through R-C, at a recalculated cost of 4 + 1 = 5.

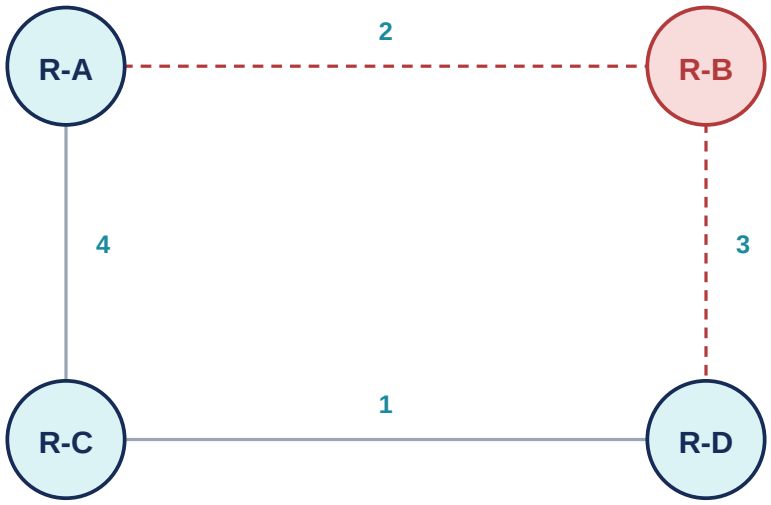

Figure 3. Network topology after Router B fails; the route automatically shifts to A → C → D.

This before/after comparison — A → B → D under normal conditions, A → C → D after R-B fails — is precisely the failover behavior that link-state protocols such as OSPF provide in production networks: when a router or link becomes unavailable, the network automatically recomputes and converges on a new shortest path without requiring manual intervention.

7. Python Implementation

The network topology is represented directly as a Python dictionary of dictionaries, where each outer key is a router and each inner dictionary maps a neighboring router to the cost of the link between them. This adjacency-list representation is the standard starting point for graph algorithms including Dijkstra's shortest-path algorithm, which the simulator's path calculator implements on top of this structure.

```
graph = {
    'A': {'B': 2, 'C': 4},
    'B': {'A': 2, 'D': 3},
    'C': {'A': 4, 'D': 1},
    'D': {'B': 3, 'C': 1}
}

def show_routes():
    for node in graph:
```


Mayank Swaraj | github.com/mayankswaraj18cr-cmd | linkedin.com/in/mayank-swaraj-4b84b6410 | mayankswaraj18cr@gmail.com

```
        print(node, "->", graph[node])

show_routes()

# Output:
# A -> {'B': 2, 'C': 4}
# B -> {'A': 2, 'D': 3}
# C -> {'A': 4, 'D': 1}
# D -> {'B': 3, 'C': 1}
```

Listing 1. Adjacency-list representation of the example network in Python.

8. Advanced Features

Beyond the core shortest-path engine, the simulator design incorporates OSPF-style shortest-path simulation as its default routing mode, an alternative RIP-style hop-count mode for comparison, graphical network topology visualization, a packet analyzer for monitoring simulated traffic load, and automatic failure recovery that recalculates affected routing table entries whenever a router is marked down.

9. Applications

The routing and failover concepts modeled here underpin operations at Internet Service Providers managing backbone networks, data centers routing internal traffic between racks and clusters, telecommunications companies operating mobile networks, cloud computing platforms coordinating server-to-server communication, and enterprise networks monitoring and maintaining internal infrastructure.

10. Result

A Dynamic Network Routing Simulator was successfully developed to simulate routers, routing tables, shortest-path calculations, and network failure recovery. The system correctly computed the optimal route between routers under normal conditions and automatically recalculated an alternative route when a router was marked as failed, demonstrating the core mechanics of real-world routing protocols such as OSPF. The accompanying interactive HTML tool reproduces this behavior, including live failure simulation and dynamic routing table recalculation.

References

- Kurose, J. F., and Ross, K. W. *Computer Networking: A Top-Down Approach*, 8th Edition. Pearson, 2020.
- Moy, J. "OSPF Version 2." RFC 2328, Internet Engineering Task Force, 1998.
- Hedrick, C. "Routing Information Protocol." RFC 1058, Internet Engineering Task Force, 1988.
- Rekhter, Y., Li, T., and Hares, S. "A Border Gateway Protocol 4 (BGP-4)." RFC 4271, 2006.
- GGSIPU B.Tech Computer Science Engineering Curriculum, Semester 7 — Advanced Computer Networks & Mobile Computing course structure.

— Mayank Swaraj

Mayank Swaraj | github.com/mayankswaraj18cr-cmd | linkedin.com/in/mayank-swaraj-4b84b6410 | mayankswaraj18cr@gmail.com